

EE/CS 172/130
Digital Logic and Design

Topic 1 – Number Systems

Dr. Maria Waqas

Administrivia

Course Outline

- **Introduction to Digital Logic and Design**
- **Binary numbers:** Number-base conversions; Octal and hexadecimal numbers; Complements of numbers; Signed binary numbers; Binary codes
- **Binary logic and logic gates:** Digital logic gates and Boolean functions; Boolean algebra; Canonical and standard forms; Karnaugh maps (2,3 and 4 variables); NAND/NOR implementations
- **Combinational Circuits:** Design and analysis procedures; Adders and subtractors; Decoders; Encoders; Multiplexers; Magnitude comparator
- **Programmable logic devices and integrated circuits**
- **Sequential Circuits:** Storage elements – latches and flip flops; Design and analysis of clocked sequential circuits; Finite state machines (FSM) - Mealy and Moore; State reduction in FSMs by implication charts; Registers and shift registers; Synchronous and asynchronous counters

Administrivia

Course Learning Outcomes

- By the end of the course, you will be able to:
- **CLO 1:** Apply r-base (binary, octal, decimal, hexadecimal numbers systems) to digital systems and carry out arithmetic operations and conversions.
- **CLO 2:** Apply principles of Boolean Algebra to represent and build equivalent realizations of digital logic (circuits).
- **CLO 3:** Design combinational logic circuits using logic gates.
- **CLO 4:** Design sequential systems using finite state machine methodology.

Administrivia

Text Book

- Title: **Digital Design**
- Authors: M. Morris Mano and Michael D. Ciletti
- Edition: 5th
- Publisher: Pearson
- Publication Date: January 1, 2012
- ISBN: 978-8131794746

Administrivia

Assessments

- Quizzes: **40%** (5 quizzes of 7 marks each)
- Assignments: **10%** (4 assignments of 2.5 marks each)
- Midterm Exam: **25%**
- Course Project: **20%**
- Class participation: **5%**
- **Late Submission Policy:** Late submissions will be penalized as follows:
 - 1-day late submission – 20% deduction of the maximum allowable marks
 - 2-days late submission – 40% deduction of the maximum allowable marks
 - No submission will be accepted after two days of the original deadline.
- **Final Exam:** There is no final written exam but the final examination day as scheduled for this course will be used for **course project presentation and evaluation**.

GRADING SCALE		
LETTER GRADE	GPA POINTS	PERCENTAGE
A+	4.00	[95 – 100]
A	4.00	[90 – 95)
A-	3.67	[85 – 90)
B+	3.33	[80 – 85)
B	3.00	[75 – 80)
B-	2.67	[70 – 75)
C+	2.33	[67 – 70)
C	2.00	[63 – 67)
C-	1.67	[60 – 63)
F	0.00	[0 – 60)

Administrivia

Instructor

- Name: **Dr. Maria Waqas**
- Email: maria.waqas@sse.habib.edu.pk
- Consultation timings: 16:00 hrs to 17:00 hrs on Tuesdays
- Office location: F-Pod C-203 Meeting Room

Why Study Digital Logic and Design?

Real-World Applications

- Logic circuits are at the heart of smartphones, laptops, IoT devices, medical equipment, and every modern electronic system.

Foundation of Computing

- All software runs on hardware; digital logic explains how 0s and 1s are processed inside a computer.
- Concepts like memory, processors, registers, and instruction execution are built on digital logic.
- Helps you understand how programming languages, compilers, and operating systems interact with the processor.
- Knowing hardware limitations (speed, memory, energy) helps you write better software.

System Software Connection

- Understanding digital logic helps in writing compilers, operating systems, device drivers, and other system software.
- It builds the bridge between software instructions and the hardware execution beneath them.

Why Study Digital Logic and Design?

Problem-Solving and Design Thinking

- Trains you to design reliable systems, from simple gates to complex processors.

Industry Relevance

- Core for careers in system software development, computer architecture, embedded systems, VLSI design, robotics, and AI hardware (GPUs/TPUs).
- Hands-on with tools like Verilog, and VHDL, giving you practical skills valued in academia and industry.

Electronic Circuits and Systems

- Electronic circuits and systems are classified as being analog or digital.

Analog Systems

- Systems in which signals (voltage or current) vary continuously over a range of values.
- **Examples:**
 - Output from a microphone or audio amplifier
 - Pressure/temperature sensor readings
 - Analog voltmeter or ammeter (pointer-based instruments)
- **The world is analog:** most physical quantities like sound, light, and temperature are inherently continuous.

Digital Systems

- Systems in which signals (voltage or current) take on only a finite number of discrete values.
- Most common digital systems are **Binary systems** (0 and 1), also called **switching circuits**.
- Transition times between discrete states are very brief and not considered useful output.
- **Examples:**
 - Computers, smartphones, digital watches, calculators.

Why Binary Digital Systems?

- Binary system uses just two symbols (0 and 1); hence the device that processes binary signals are very simple.
 - A simple electronic switch with two states: **On** and **OFF** (**HIGH** and **LOW**, **1** and **0**, **TRUE** and **FALSE**)
 - Logic operations are performed by transistors in processors or field programmable gate arrays (FPGAs).
- Well suited for both numerical and non-numerical information.
- Can be processed and transmitted more efficiently and reliably.
 - Are quite insensitive to variations of component parameter values (such as operating temperatures).
 - The physical representation of values are sufficiently separated, especially in binary systems, so that small variations do not change the value.
- Storage efficient
 - For example, music when converted to digital form can be stored and reproduced with greater accuracy and clarity.

Hybrid Systems

- Combination of both analog and digital.
- **Example:**



Common Number Systems in Computing

Decimal

- Base: 10
- Digits: 0–9
- Used in everyday life and by humans for input/output.
- Example: 245_{10}
- Counting: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ...19, 20, 21, 22, ...99, 100, 101, 102,

Binary

- Base: 2
- Digits: 0, 1
- Fundamental to computers, since digital systems operate on two states (ON/OFF, High/Low).
- Example: 1011_2
- Counting: 0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111,

Common Number Systems in Computing

Octal

- Base 8
- Digits: 0–7
- Used as a shorthand for binary, since 1 octal digit = 3 binary bits.
- Example: 376_8
- Counting: 0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13,17, 20, 21,777, 1000, 1001,

Hexadecimal

- Base: 16
- Digits: 0–9, A–F (where A=10, B=11, C=12, D=13, E=14, F=15).
- Very common in computer architecture, memory addresses, assembly language, and debugging.
- Compact representation of binary (1 hex digit = 4 binary bits).
- Example: $4F_{16}$
- Counting: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11, 12, 13,1F, 20, 21,FF, 100, 101, 102,

Common Number Systems in Computing

General Rule

- Base: r
- Digits: 0 to $(r-1)$, all the digits (coefficients) fall in this range
- Total digits: r
- Nomenclature: number_r or $(\text{number})_r$
- Weights: increase from 0 onwards for digits on left of the radix point and decrease from -1 onwards on right of radix point.

Conversion Between Denary and Other Number Systems

To Denary

- Consider a base r number in the following form:

$$\dots a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} \dots$$

- A number expressed in a base- r system has coefficients multiplied by powers of r :

$$a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_2 \cdot r^2 + a_1 \cdot r^1 + a_0 + a_{-1} \cdot r^{-1} + a_{-2} \cdot r^{-2} + \dots + a_m \cdot r^m$$

Examples

- $123.45)_{10}$

$$1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} = 100 + 20 + 3 + 0.4 + 0.05 = 123.45)_{10}$$

- $110.11)_2$

$$1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 4 + 2 + 0 + 0.5 + 0.25 = 6.75)_{10}$$

- $126.1)_8$

$$1 \times 8^2 + 2 \times 8^1 + 6 \times 8^0 + 1 \times 8^{-1} = 64 + 16 + 6 + 0.125 = 86.125)_{10}$$

- $3A.1)_{16}$

$$3 \times 16^1 + A \times 16^0 + 1 \times 16^{-1} = 48 + 10 + 0.0625 = 58.0625)_{10}$$

Conversion Between Denary and Other Number Systems

From Denary

- Consider a decimal number in the following form:

$$\dots a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} \dots$$

- The conversion of a decimal integer to a number in base r is done in two parts:
 - Divide the integer part (left hand side) of the number and all its successive quotients by r and accumulating the remainders until the quotient becomes 0.
 - Multiply the fraction part (right hand side) of the number with base r , accumulate the integer part of the resultant value and use its fraction part to multiply with base r again. Process continues until either the fraction becomes 0 or the number of digits has sufficient accuracy.

Examples

- 6.75_{10} to binary
 $110_2 + 0.11_2 = 110.11_2$
- 58.0625_{10} to hexadecimal
 $3A + 0.1 = 3A.1_{16}$
- 86.125_{10} to octal
 $126 + 0.1 = 126.1_8$

Activity

- Write down your HU ID, then write down the last digit of your HU ID that is not 0 or 1 or 2.
- This number is your base or radix. Write down all coefficients/digits that are associated with it.
- Next write down the first 15 numbers of your number system.
- Write down any 3 digit random number of your base using your coefficients and find out its value in decimal.
- You will find that your number system exists out there! Do some research and find out its name.
- Consider your HU ID to be a base 10 number. Convert it directly to binary and hexadecimal.

Conversion Between Binary and Octal

From Binary to Octal

- An octal digit is represented by 3 binary bits (since $2^3 = 8$).
- Group binary digits in sets of 3 bits.
 - Start from the radix point and move outwards.
 - Pad with zeros if needed to complete groups.
- Replace each 3-bit group with its octal equivalent.

Examples

- $101110.011)_2$
 $= 56.3)_8$
- $11110.11)_2$
 $= 36.6)_8$

Conversion Between Binary and Octal

From Octal to Binary

- Expand each octal digit into a 3-bit binary group.
 - Integer and fractional parts are handled separately and then joined.
- Concatenate the groups to get the binary number.

Examples

- $56.3)_8$
 $= 101110.011)_2$
- $36.6)_8$
 $= 011110.110)_2 = 11110.11)_2$

Conversion Between Binary and Hexadecimal

From Binary to Hexadecimal

- A hexadecimal digit is represented by 4 binary bits (since $2^4 = 16$).
- Group binary digits in sets of 4 bits.
 - Start from the radix point and move outwards.
 - Pad with zeros if needed to complete groups.
- Replace each 4-bit group with its hexadecimal equivalent.

Examples

- $10101000.0101)_2$
= $A8.5)_{16}$
- $11110.11)_2$
= $1E.C)_{16}$

Conversion Between Binary and Octal

From Hexadecimal to Binary

- Expand each hexadecimal digit into a 4-bit binary group.
 - Integer and fractional parts are handled separately and then joined.
- Concatenate the groups to get the binary number.

Examples

- $A8.5)_{16}$
 $= 10101000.0101)_2$
- $1E.C)_{16}$
 $= 00011110.1100)_2 = 11110.11)_2$

Summarizing Conversions

Denary	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Denary	Binary	Octal	Hex
8	1000	10	8
9	10001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Binary Numbers

- A binary digit is called a **bit**, represented by **b**.
- Group of 4 bits is called a **nibble**.
- Group of 8 bits is called a **byte**, represented by **B**.
- Number of combinations/values possible for n bits is 2^n .

Higher Units

Unit	Number of Bytes	Equivalent Denary
1 kibibyte (1KiB)	2^{10}	1024 B
1 mebibyte (1MiB)	2^{20}	1048576 B
1 gibibyte (1GiB)	2^{30}	1073741824 B
1 tebibyte (1 TiB)	2^{40}	1099511627776 B
1 pebibyte (1PiB)	2^{50}	1125899906842624 B
1 exbibyte (1EiB)	2^{60}	1152921504606846976 B

Binary Arithmetic

Binary Addition

All numbers on this slide are binary numbers

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$

- $0 + 0 + 0 = 0$
- $0 + 0 + 1 = 1$
- $0 + 1 + 0 = 1$
- $0 + 1 + 1 = 10$
- $1 + 0 + 0 = 1$
- $1 + 0 + 1 = 10$
- $1 + 1 + 0 = 10$
- $1 + 1 + 1 = 11$

- $1011 + 0010$ (4-bit addition)
= 1101

- $10110101 + 11101111$ (8-bit addition)
= 110100100

↓
Overflow

Binary Arithmetic

Binary Subtraction

- $1011 - 0010$
 $= 1001$
- $10110111 - 01101111$
 $= 01001000$
- $0010 - 1011 = ?$

Binary Multiplication

- 11×10
 $= 110$
- 1011×1111
 $= 10100101$

All numbers on this slide are binary numbers

Binary Division

- $110 \div 10$
 $= 11$
- $1100 \div 11$
 $= 100$

Complement Systems in Binary

- Types of binary complement systems: 1's complement and 2's complement system.

1's Complement Conversion

- To find the 1's complement of a binary number, invert all bits (replace 0 with 1, and 1 with 0).
- To convert a 1's complement number back to binary, invert all bits again.

Example

- $10110111_2 = 01001000_{1's}$

2's Complement Method

- To convert to 2's complement, invert all bits (form 1's complement), then add 1 to the result.
- To convert back, subtract 1, then invert all bits.

Example

- $10110111_2 = 01001000_{1's} = 01001001_{2's}$

Unsigned and Signed Binary Numbers

Unsigned Binary Numbers

- All numbers are assumed to be positive.
- Denary equivalents can be obtained through straight forward conversion methods.
- Range in denary: 0 to $(2^n - 1)$, where n is the number of bits

Example

- For 3-bit binary numbers
 - n : 3
 - Range: 0_{10} to 7_{10}
 - Denary numbers: 0, 1, 2, 3, 4, 5, 6, 7
 - Binary numbers: 000, 001, 010, 011, 100, 101, 110, 111

Unsigned and Signed Binary Numbers

Signed Binary Numbers

- Can represent both positive and negative numbers.
- Most significant bit (MSB) serves as the sign bit: 0 → positive number, and 1 → negative numbers.
- In real computer architecture, 2's complement is the standard representation for unsigned numbers.

Representation of Denary Numbers in 2's Complement System

- **For Positive Numbers:**
 - Convert the number to binary using the standard method.
 - Ensure the MSB = 0.
 - Conversion back to denary is straightforward.
- **For Negative Numbers:**
 - Find the binary equivalent of the magnitude (with MSB = 0).
 - Take the 2's complement.
 - To convert back: take the 2's complement again, convert to decimal, and place a leading minus sign.

Example

- $+5)_{10} = 0101)_{2's}$
- $-5)_{10} = 1011)_{2's}$

Negative Weight Method

- Interpret the binary number using positional weights, but give MSB a negative weight.

Unsigned and Signed Binary Numbers

Range of Denary Numbers in 2's Complement System

- Range: $-(2^{n-1})$ to $+(2^{n-1} - 1)$

Example

- For 3-bit binary numbers
 - n: 3
 - Range: -4_{10} to $+3_{10}$
 - Denary equivalents: -4, -3, -2, -1, 0, 1, 2, 3
 - Binary numbers: 100, 101, 110, 111, 000, 001, 010, 011

2's Complement Arithmetic

Rules for Arithmetic in 2's Complement

- Represent operands in 2's complement notation (with the same number of bits).
- Perform binary addition of the operands.
- Check for overflow:
 - Overflow occurs if the result is too large for the available number of bits.
 - It is detected when the two operands have the same sign, but the result has the opposite sign.
- Handle carry:
 - If overflow occurs, there is a carry out of the MSB, either report an error, or increase the bit width and repeat the operation (if possible).
 - If overflow does not occur, discard the carry

2's Complement Arithmetic

Example

Denary		Binary
5 - 4	(+5) + (-4)	0101 + 1100 = 0001 (carry discarded)
-5 + 4	(-5) + (+4)	1011 + 0100 = 1111 (carry discarded)
-5 - 4	(-5) + (-4)	1011 + 1100 = 10111 (overflow) With 5 bits: 11011 + 11100 = 10111 (no overflow)
5 + 4	(+5) + (+4)	0101 + 0100 = 1001 (overflow) With 5-bits: 00101 + 00100 = 01001 (no overflow)

- 2's complement representation eliminates the need for a separate subtractor in computing system hardware.
- The same binary adder circuit can be used for both addition and subtraction operations.

Other Signed Systems

- Apart from 2's complement system, other less frequently used systems are sign magnitude and 1's complement system.

Example

- For $n=3$, range: -4_{10} to $+3_{10}$

Decimal	Sign Magnitude	1's Complement	2's Complement
-4	---	---	100
-3	111	100	101
-2	110	101	110
-1	101	110	111
-0	100	111	000
+0	000	000	
1	001	001	001
2	010	010	010
3	011	011	011